

Capstone Task: Build, Test and Deploy a Secure E-Commerce REST API

Issued by: Fonyuy Gita
Stack: Node.js · Express.js · MongoDB (Mongoose) · JWT
Type: Individual project (build, test, deploy)
Suggested duration: 3–4 weeks

1. Introduction

You have completed the MongoDB tutorial. This task is where you prove you can use it in a real system. You will build a **production-style e-commerce backend API** that covers the full customer journey: registering, logging in, browsing products, adding to a cart, placing an order, paying, and receiving confirmation. You will also build the admin tools needed to run the shop.

There is **no frontend** in this task. Your API will be tested with Postman, Thunder Client, or automated tests. The focus is on **correct design, clean code, security, testing and deployment**.

Important: Security is the most heavily weighted part of this project. A working API that is insecure will not pass.

2. Learning Objectives

- By the end of this project you should be able to:
- 1. Design MongoDB schemas and relationships for a real business domain.
 - 2. Build a layered Express application (routes → controllers → services → models).
 - 3. Implement authentication and role-based authorization with JWT.
 - 4. Protect an API against common attacks (OWASP Top 10 for APIs).
 - 5. Integrate a third-party payment provider and handle webhooks safely.
 - 6. Write automated unit and integration tests.
 - 7. Deploy the API to the cloud with environment-based configuration.
 - 8. Document your API so another developer can use it without asking you questions.

3. Technical Requirements

Area	Requirement
Runtime	Node.js (LTS version)
Framework	Express.js
Database	MongoDB with Mongoose (use MongoDB Atlas for deployment)

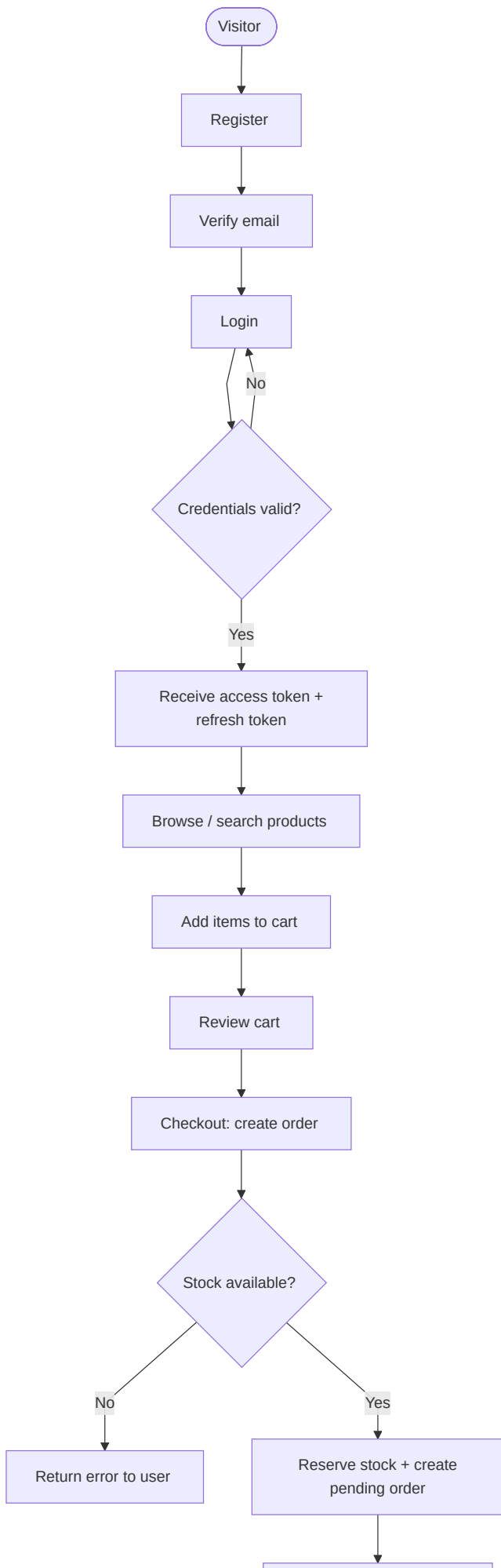
Area	Requirement
Authentication	JWT (access token + refresh token)
Password hashing	<code>bcrypt</code> or <code>argon2</code>
Validation	<code>Joi</code> , <code>Zod</code> or <code>express-validator</code>
Testing	<code>Jest</code> + <code>Supertest</code> (and <code>mongodb-memory-server</code> for the test DB)
Payment	Stripe (test mode) or a local provider sandbox such as Flutterwave / Campay / Notch Pay
Documentation	Swagger/OpenAPI or a complete Postman collection
Version control	Git + GitHub, with meaningful commits
Deployment	Render, Railway, Fly.io, or a VPS (your choice)

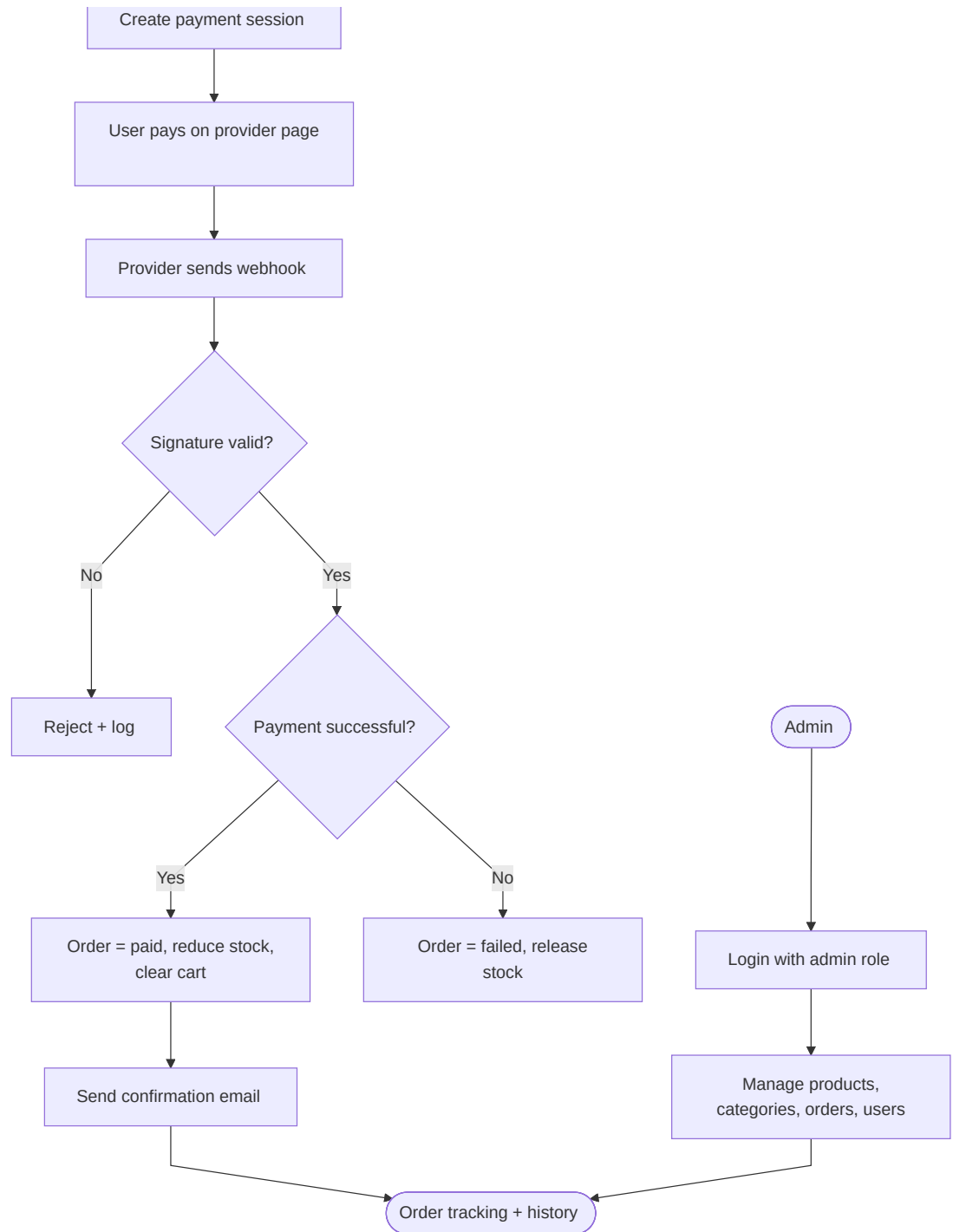
Recommended folder structure

```
src/
  config/      # env, db connection
  models/     # Mongoose schemas
  routes/     # route definitions only
  controllers/ # request/response handling
  services/   # business logic
  middlewares/ # auth, roles, validation, error handler, rate limit
  utils/     # helpers (AppError, token utils, logger)
  validators/ # request schemas
  tests/     # unit + integration tests
app.js
server.js
```

4. Project Flow

4.1 Overall customer and admin journey





The flow in plain English

Think of your API as an online shop with a front door, a shopping aisle, a till and a back office.

1. **Getting in.** A visitor creates an account (register) and confirms their email address. After that they log in with their email and password. If the details are wrong, they are sent back to try again. If they are right, the API hands them two "passes" (tokens) that prove who they are on every later request.
2. **Shopping.** With a valid pass, the customer can browse and search products, then add the ones they want to their cart. They can review the cart and change it as much as they like. Nothing is bought yet.
3. **Placing the order.** When the customer checks out, your API checks that every item is still in stock. If something has run out, the customer gets an error and the order stops there. If all is well, the API reserves the stock and creates an order marked **pending**, meaning "waiting for payment".
4. **Paying.** The API asks the payment provider to open a payment session, and the customer pays on the

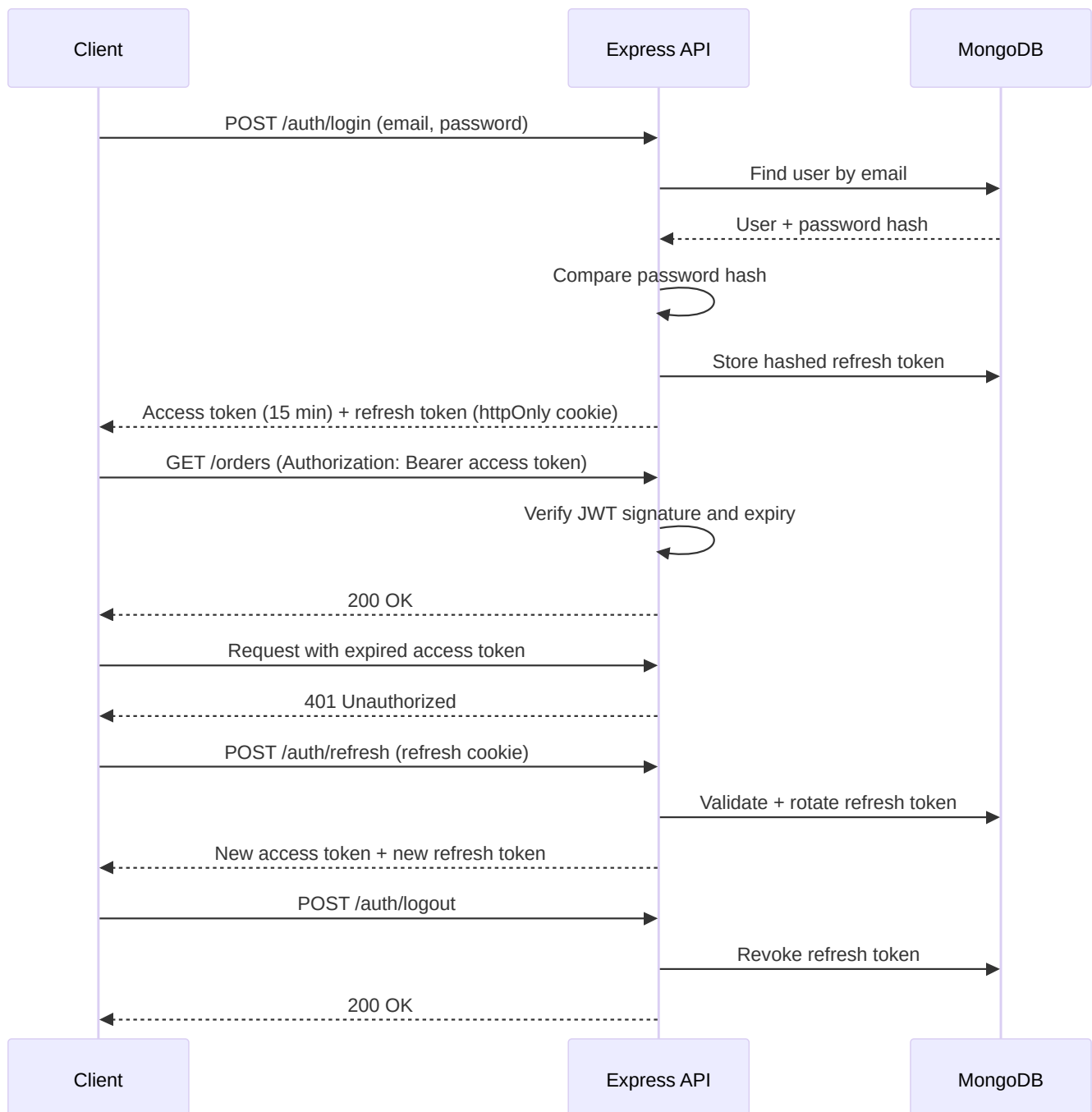
provider's page. Your API never sees card details.

5. **Confirming the payment.** The payment provider sends your API a message (a webhook) saying what happened. Your API first checks that the message is genuine, using a secret signature. A fake message is rejected and logged. A genuine one is read: if the payment succeeded, the order becomes **paid**, stock is permanently reduced, the cart is emptied and a confirmation email goes out. If the payment failed, the order becomes **failed** and the reserved stock is released for other customers.

6. **After the sale.** The customer can see their order history and track each order.

The **admin** follows a separate path. They log in with an admin account and can manage products, categories, orders and users. Their work feeds back into what customers see and how orders progress.

4.2 Authentication flow (access + refresh tokens)



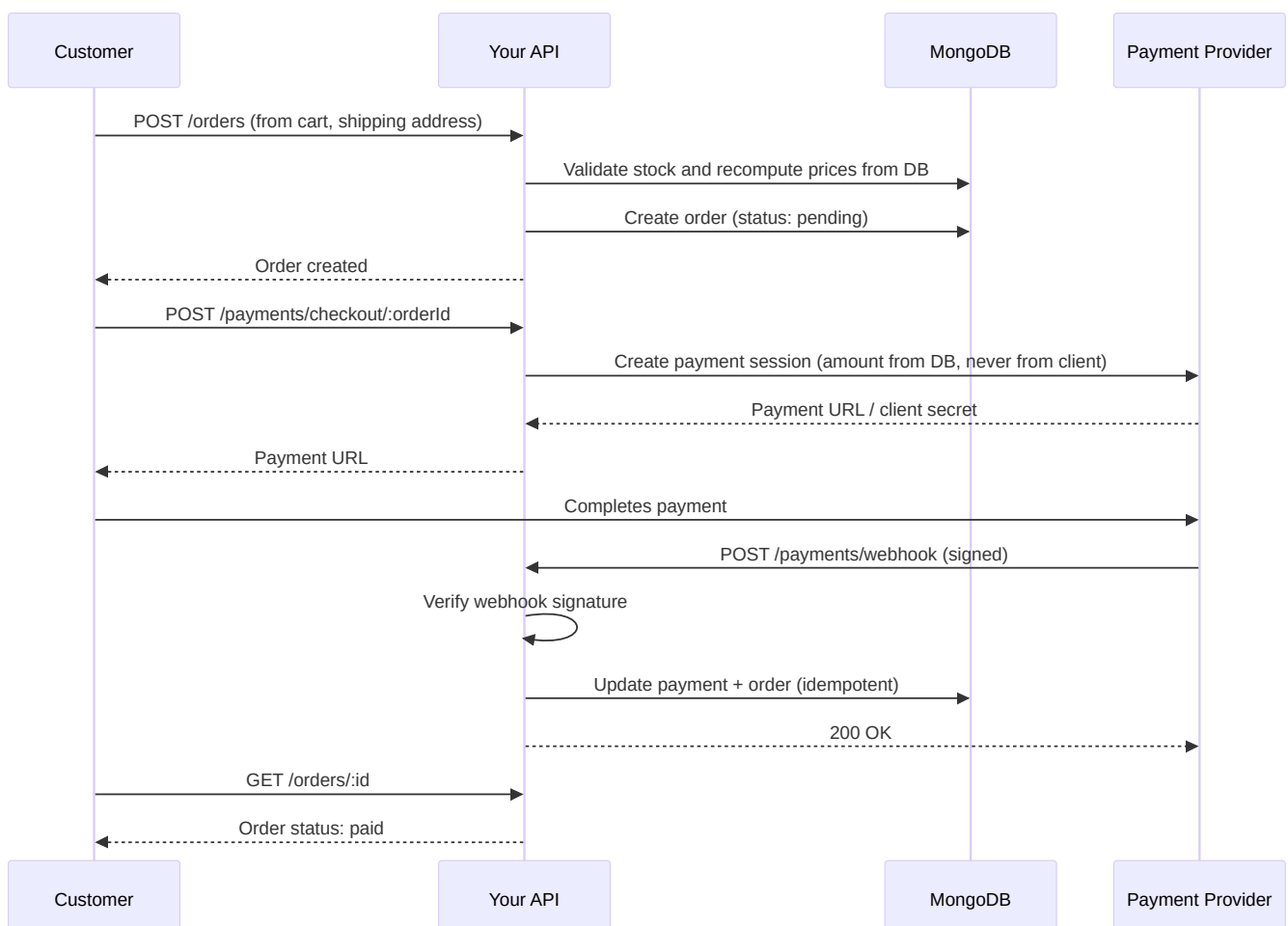
The flow in plain English

Think of this like a cinema wristband and a membership card.

1. The user sends their email and password to the login endpoint. The API finds that user in the database and compares the password they typed with the stored **hash** (the scrambled version of the password). The real password is never stored.
2. If it matches, the API gives back two things: a short-lived **access token** (valid for about 15 minutes, like a wristband for today) and a longer-lived **refresh token** (like a membership card used to get a new wristband). A hashed copy of the refresh token is saved in the database so the server can recognise it later.
3. For every normal request, such as viewing orders, the client shows the access token. The API checks that the signature is real and that it has not expired. If both are fine, the request goes through.
4. After 15 minutes the access token expires and the API answers `401 Unauthorized`. The client then quietly sends the refresh token to the refresh endpoint. The API checks it against the database, then issues a **brand new pair** of tokens and invalidates the old refresh token. This is called *rotation*.
5. On logout, the API deletes or revokes the refresh token, so nobody can use it again.

Why it is built this way: if someone steals an access token, it stops working in minutes. The refresh token is the valuable one, so it is kept in a protected cookie, stored hashed on the server, and replaced on every use.

4.3 Checkout and payment flow



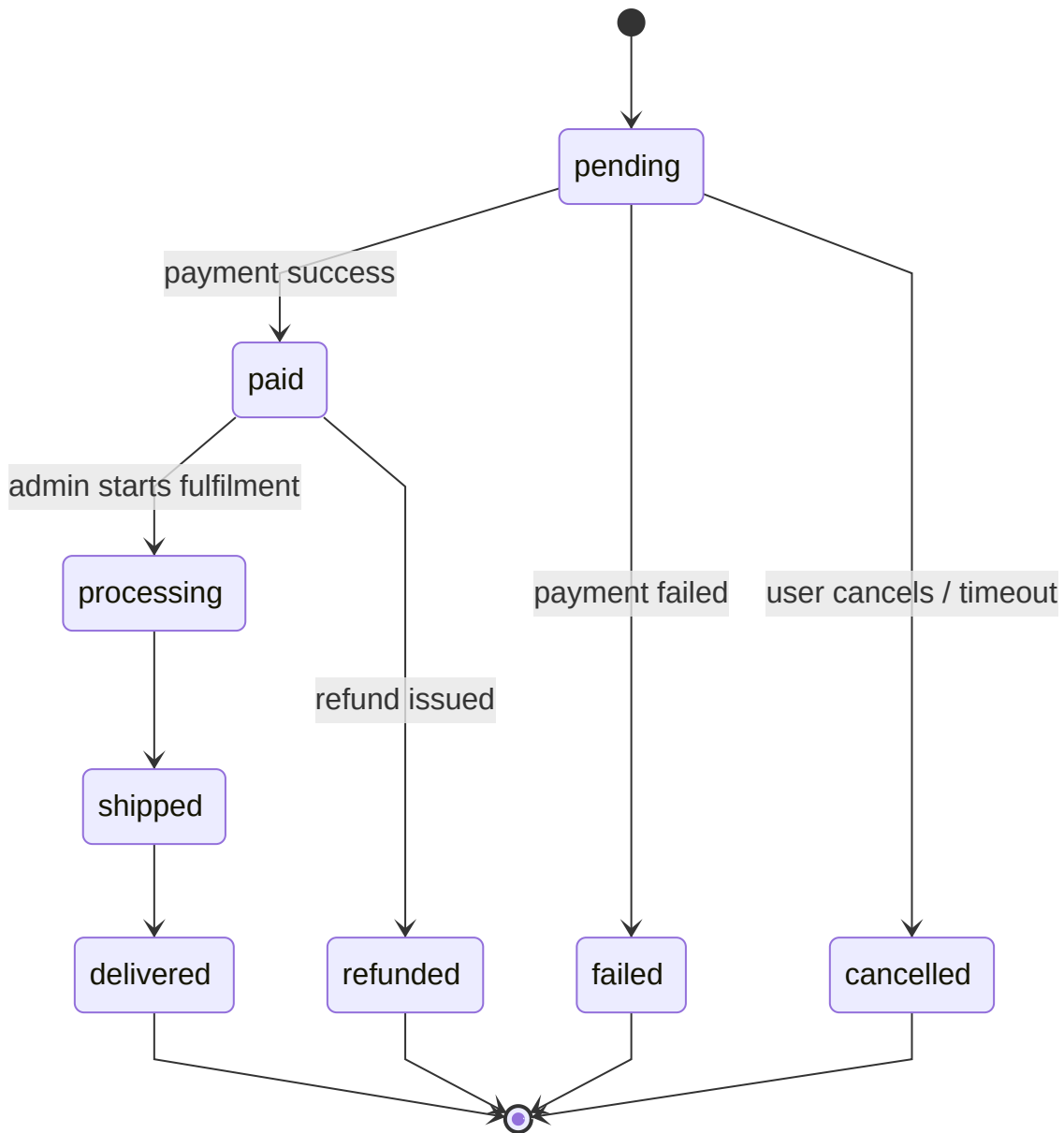
The flow in plain English

This diagram shows the most sensitive part of the system: turning a cart into money.

1. The customer asks to create an order from their cart and gives a shipping address.
2. Your API does **not** trust the prices the client might send. It looks up every product in the database, checks stock, and works out the total itself. Then it saves the order with the status **pending**.
3. The customer asks to pay for that order. Your API again takes the amount **from its own database** and asks the payment provider to create a payment session. The provider replies with a payment link, which your API passes to the customer.
4. The customer pays on the provider's own page. Your server never touches card numbers.
5. When the payment is done, the provider calls your **webhook** endpoint with a signed message. Your API verifies the signature to be sure it really came from the provider, then updates the payment and order records. It must do this in a way that is safe if the same message arrives twice (idempotent), so an order is never processed two times.
6. Your API replies `200 OK` to the provider to say "message received". Later, when the customer checks the order, they see the status is **paid**.

The key idea: the only thing that can mark an order as paid is the verified message from the payment provider. The customer's browser or app saying "I paid" counts for nothing.

4.4 Order status lifecycle



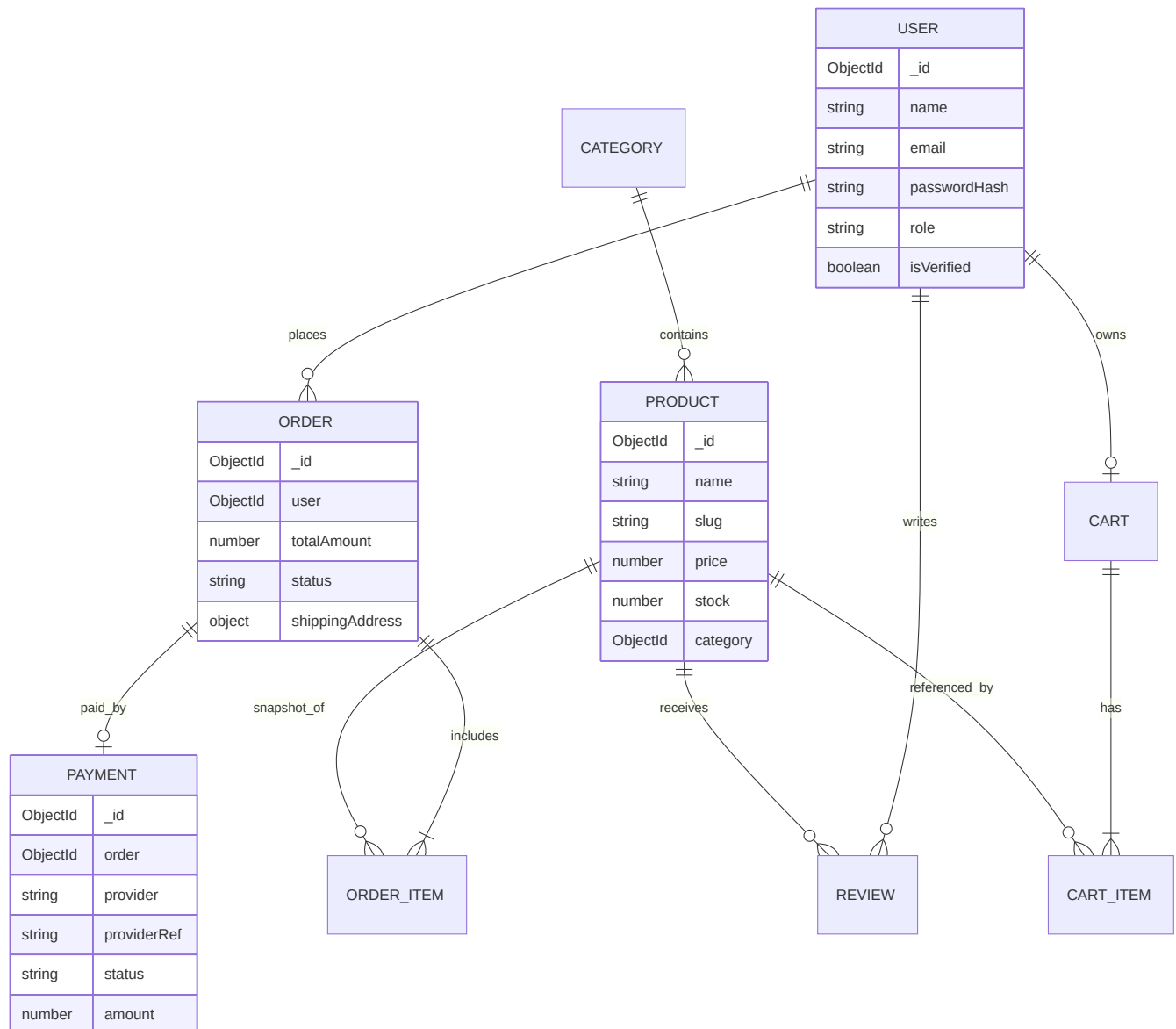
The flow in plain English

An order is like a parcel moving through stations, and it can only move in certain directions.

- Every order starts as **pending**: created, but not yet paid.
- From pending it can go three ways: **paid** if the payment succeeds, **failed** if the payment is declined, or **cancelled** if the customer changes their mind or the 30-minute timer runs out. Failed and cancelled are dead ends, and the reserved stock goes back on the shelf.
- Once **paid**, only an admin moves it forward: **processing** (being packed), then **shipped**, then **delivered**, which is the happy ending.
- A paid order can also be **refunded** if something goes wrong, for example a damaged item.

Why this matters: your code must enforce these rules. A customer must not be able to cancel a shipped order, and nobody should be able to jump from pending straight to delivered. Any move not shown in the diagram should be rejected.

4.5 Data model relationships



The flow in plain English

This diagram shows what data you store and how the pieces connect. Read each line as a sentence.

- A **user** can place many **orders**, owns one **cart**, and can write many **reviews**.
- A **category** contains many **products**, and a **product** can receive many **reviews**.
- A **cart** holds several **cart items**, and each cart item points to a product.
- An **order** includes several **order items**. Each order item is a *snapshot* of the product at the time of purchase (name and price), so changing a product's price tomorrow does not rewrite yesterday's orders.
- An **order** is settled by one **payment** record, which stores the provider's reference, the amount and the status.

The boxes list the most important fields for each collection. For example, a user stores a `passwordHash` (never the password) and a `role` (customer or admin). A product stores its `price` and `stock`. An order stores its `status` and `totalAmount`. In MongoDB you will decide which of these relationships are done by **referencing** another document's `_id` and which by **embedding** data inside a document. You will be asked to justify those

choices in the review.

5. Functional Requirements and Endpoints

Base URL: `/api/v1`. All responses must use a consistent JSON format:

```
{ "success": true, "message": "Product created", "data": { } }
{ "success": false, "message": "Validation failed", "errors": [ ] }
```

5.1 Authentication and users

Method	Endpoint	Access	Description
POST	<code>/auth/register</code>	Public	Create account (role is always <code>customer</code>)
POST	<code>/auth/verify-email/:token</code>	Public	Verify email address
POST	<code>/auth/login</code>	Public	Login, receive tokens
POST	<code>/auth/refresh</code>	Public (refresh token)	Get new access token
POST	<code>/auth/logout</code>	Auth	Revoke refresh token
POST	<code>/auth/forgot-password</code>	Public	Send reset link
POST	<code>/auth/reset-password/:token</code>	Public	Set new password
GET	<code>/users/me</code>	Auth	View own profile
PATCH	<code>/users/me</code>	Auth	Update own profile
PATCH	<code>/users/me/password</code>	Auth	Change password (requires current password)
GET	<code>/users</code>	Admin	List users (paginated)
PATCH	<code>/users/:id/role</code>	Admin	Change a user's role
DELETE	<code>/users/:id</code>	Admin	Deactivate a user

5.2 Categories and products

Method	Endpoint	Access	Description
GET	<code>/categories</code>	Public	List categories
POST	<code>/categories</code>	Admin	Create category

Method	Endpoint	Access	Description
PATCH	/categories/:id	Admin	Update category
DELETE	/categories/:id	Admin	Delete category
GET	/products	Public	List with pagination, filtering, sorting, search
GET	/products/:id	Public	Product details
POST	/products	Admin	Create product (with image upload)
PATCH	/products/:id	Admin	Update product
DELETE	/products/:id	Admin	Soft-delete product

Example query: `GET /products?category=shoes&minPrice=50000&maxPrice=300000&sort=-price&page=2&limit=10&search=nike`

5.3 Cart

Method	Endpoint	Access	Description
GET	/cart	Customer	View own cart
POST	/cart/items	Customer	Add item (<code>productId</code> , <code>quantity</code>)
PATCH	/cart/items/:productId	Customer	Change quantity
DELETE	/cart/items/:productId	Customer	Remove item
DELETE	/cart	Customer	Clear cart

5.4 Orders

Method	Endpoint	Access	Description
POST	/orders	Customer	Create order from cart
GET	/orders	Customer	Own order history
GET	/orders/:id	Owner / Admin	Order details
PATCH	/orders/:id/cancel	Owner	Cancel while status is <code>pending</code>
GET	/admin/orders	Admin	All orders with filters
PATCH	/admin/orders/:id/status	Admin	Update fulfilment status

5.5 Payments

Method	Endpoint	Access	Description
POST	<code>/payments/checkout/:orderId</code>	Owner	Create payment session
POST	<code>/payments/webhook</code>	Provider only	Receive payment events (signature verified)
GET	<code>/payments/:orderId</code>	Owner / Admin	Payment status
POST	<code>/payments/:orderId/refund</code>	Admin	Refund a payment

5.6 Reviews (bonus feature)

Method	Endpoint	Access	Description
POST	<code>/products/:id/reviews</code>	Customer who bought it	Add review
GET	<code>/products/:id/reviews</code>	Public	List reviews

6. Business Rules (must be enforced in code)

1. **Prices are never trusted from the client.** Always read price and stock from the database when creating an order.
2. Stock must never go negative. Use atomic updates (`$inc` with a condition) or MongoDB transactions to prevent overselling when two users buy the last item at the same time.
3. Store a **snapshot** of product name and price inside each order item, so later price changes do not alter past orders.
4. A user can only view, cancel or pay for **their own** orders.
5. Only `pending` orders can be cancelled or paid.
6. Payment webhooks must be **idempotent**: receiving the same event twice must not process the order twice.
7. Orders that stay `pending` for more than 30 minutes should be cancelled and stock released (a simple scheduled job is enough).
8. Money is stored in the smallest unit (e.g. integers for XAF, cents for USD), never as floating point decimals.

7. Security Requirements (Critical)

Every item below is **mandatory** and will be checked during review.

7.1 Authentication and passwords

- Hash passwords with `bcrypt` (cost 12 or higher) or `argon2`. Never store or log plain text passwords.
- Enforce a strong password policy (minimum length 8, mix of characters).
- Access tokens expire in **15 minutes or less**. Refresh tokens last days, are stored **hashed** in the database, and are **rotated** on each use.
- Send refresh tokens in an `httpOnly`, `secure`, `sameSite` cookie.
- JWT secrets must be long, random and stored in environment variables. Use different secrets for access and refresh tokens.
- Specify the algorithm when verifying JWTs, and reject tokens with `alg: none`.
- Return the **same generic message** for wrong email and wrong password (`"Invalid credentials"`) to prevent user enumeration.
- Lock or slow down accounts after repeated failed logins.

7.2 Authorization

- Implement role-based access control (`customer`, `admin`) through a reusable middleware.
- Prevent **IDOR / broken object-level authorization**: always check that the resource belongs to the logged-in user.
- Prevent **mass assignment**: never pass `req.body` directly into `create()` or `update()`. A user must not be able to send `"role": "admin"` during registration.

7.3 Input handling

- Validate and sanitize **every** request body, query and parameter with a validation library.
- Prevent **NoSQL injection**: use `express-mongo-sanitize` and never build queries from raw user objects (e.g. `{ "email": { "$ne": null } }`).
- Limit request body size (`express.json({ limit: "10kb" })`).
- Validate file uploads by type and size, and never trust the original file name.
- Validate `ObjectId` parameters before querying.

7.4 HTTP and transport security

- Use `helmet` for secure headers.
- Configure `cors` with an explicit list of allowed origins, not `*`.
- Add **rate limiting** (`express-rate-limit`), with stricter limits on `/auth/login`, `/auth/register` and `/auth/forgot-password`.
- Use `hpp` to prevent HTTP parameter pollution.
- Enforce HTTPS in production.

7.5 Payment security

- **Never** handle raw card data. Use the provider's hosted checkout or client-side SDK.

- Verify the webhook **signature** using the provider's secret before doing anything.
- Confirm the paid amount and currency match the order in your database.
- Never mark an order as paid because the frontend or user says so. **Only the verified webhook (or a server-side verification call) can do that.**

7.6 Errors, logging and secrets

- Use a global error handler. In production, never return stack traces or internal error messages.
- Never log passwords, tokens or full payment payloads. Use a logger such as `winston` or `pino`, with `morgan` for HTTP logs.
- No secrets in Git. Use `.env`, add it to `.gitignore`, and provide a `.env.example`.
- Run `npm audit` and fix critical vulnerabilities.
- Create MongoDB users with the **least privilege** needed, and restrict IP access on Atlas where possible.

8. Testing Requirements

1. **Unit tests** for services and utilities (price calculation, token generation, stock logic).
2. **Integration tests** with Supertest for every major flow: register → login → add to cart → order → payment webhook.
3. **Security tests**, including at least:
 - Accessing a protected route without a token returns `401`.
 - A customer accessing an admin route returns `403`.
 - User A cannot read User B's order.
 - Registration with `"role": "admin"` does not create an admin.
 - NoSQL injection payloads are rejected.
 - A webhook with an invalid signature is rejected.
 - Overselling: two simultaneous orders for the last item, only one succeeds.
4. Use a separate test database. Never run tests against production data.
5. Minimum **70% code coverage**. Show the coverage report in your README.
6. Provide a Postman collection (or Swagger UI) containing every endpoint with example requests.

9. Deployment Requirements

1. Deploy the API to a public URL (Render, Railway, Fly.io or a VPS).
2. Use **MongoDB Atlas** for the production database.
3. All configuration comes from environment variables (`PORT`, `MONGO_URI`, `JWT_ACCESS_SECRET`, `JWT_REFRESH_SECRET`, `PAYMENT_SECRET`, `WEBHOOK_SECRET`, `CLIENT_URL`, `NODE_ENV`).
4. Add a `GET /health` endpoint.
5. Register your production webhook URL with the payment provider (test mode is fine).

6. Bonus: a `Dockerfile`, and a GitHub Actions workflow that runs tests on every push.
7. Your README must explain how to run the project locally and how it was deployed.

10. Milestones

Week	Goal
1	Project setup, schemas, authentication, roles, error handling
2	Categories, products, cart, orders, validation
3	Payment integration, webhooks, security hardening
4	Testing, documentation, deployment, final review

Commit at least once per working day. Your commit history is part of the evaluation.

11. Deliverables

1. GitHub repository link (clean structure, `.gitignore`, `.env.example`).
2. Live API URL.
3. `README.md` with setup steps, environment variables, architecture summary and the list of endpoints.
4. Postman collection or Swagger documentation.
5. Test coverage report.
6. A short **security notes** document (one page) explaining what protections you added and why.
7. A 5–10 minute demo (live or recorded) of the full flow: register → order → pay → track.

12. Evaluation Criteria

Criteria	Weight
Security implementation	25%
Correctness of business logic and endpoints	20%
Code structure and quality	15%
Testing	15%
Payment integration	10%
Deployment	10%
Documentation	5%

Automatic deductions: committed secrets or `.env`, passwords stored in plain text, prices accepted from the client, missing authorization checks, copied code you cannot explain.

Bonus (up to +10%): refresh token rotation with reuse detection, Docker, CI pipeline, Redis caching, email notifications, API pagination with cursors, audit logs.

13. Questions to Expect During the Review

Be ready to answer these **without notes**. I will ask them while you walk me through your code.

Database and design

1. Why did you choose to embed some data (e.g. order items) and reference other data (e.g. user in order)? What are the trade-offs?
2. Why do you store a price snapshot in the order instead of only referencing the product?
3. Which fields did you index, and why? What happens to search speed without those indexes?
4. How do you prevent two customers from buying the last item at the same time? What is the difference between an atomic update and a transaction?
5. Why store money as integers rather than decimals?

Authentication and authorization

6. What is the difference between authentication and authorization? Show me where each happens in your code.
7. Why use both an access token and a refresh token? What happens if an access token is stolen? What if a refresh token is stolen?
8. Why did you hash the refresh token in the database?
9. Where do you store tokens on the client and what are the risks of `localStorage` versus `httpOnly` cookies?
10. What is the difference between hashing and encryption? Why is bcrypt preferred over SHA-256 for passwords?
11. What is an IDOR vulnerability? Show me how your API prevents it.
12. What is mass assignment, and how does your registration endpoint avoid it?

Security

13. What is NoSQL injection? Show an example payload and how you defend against it.
14. What do `helmet`, `cors`, rate limiting and `hpp` each protect against?
15. Why is `Access-Control-Allow-Origin: *` dangerous for an authenticated API?
16. What is the difference between XSS, CSRF and injection attacks? Which apply to your API?
17. Why must you return the same error message for a wrong email and a wrong password?
18. How would you detect and respond to a brute-force attack on your login route?
19. What are the top risks in the OWASP API Security Top 10, and which ones did you address?

Payments

20. Why can't you trust the frontend to say a payment succeeded?
21. What is a webhook, and how do you verify it really came from the payment provider?
22. What does **idempotency** mean, and how did you make your webhook handler idempotent?
23. What happens if the customer pays but your server is down when the webhook arrives?
24. Why should the amount be read from your database rather than the request?

Testing and deployment

25. What is the difference between a unit test and an integration test? Give an example of each from your project.
26. How do you keep your tests from touching production data?
27. Which environment variables does your app need, and why should they never be committed?
28. What is different between your development and production configuration?
29. If your production database were compromised tomorrow, what damage could the attacker do, and how have you limited it?
30. If you had two more weeks, what would you improve first, and why?

14. Submission Rules

- Work **individually**. You may research and use documentation, but you must understand and be able to explain every line you submit.
- Use of AI tools for learning is fine; submitting code you cannot explain is not.
- Submit the repository link, live URL and demo before the deadline.
- Late submissions lose 5% per day.

If something in this document is unclear, ask early. Good engineers ask good questions.

Good luck, build it like it is going into production.

Fonyuy Gita